SKIP LOCKED limits the selection to rows that are not currently locked.

For a query running repeatedly, skipping locked rows would eventually select all rows, assuming that lock durations are small and the transactions holding them complete successfully.

SKIP LOCKED is a good concurrency control mechanism, when using a database as a queue of items to process.

As mentioned in *Selecting for Share and Update in PostgreSQL*,[20] rows on a referenced table by a Foreign Key column will also be locked when SELECT ... FOR UPDATE is used. This is necessary to maintain referential integrity.

Another type of weaker lock is SELECT ... FOR SHARE. This creates a shared lock on the rows being selected. A shared lock allows another concurrent reader to select the same row.

A shared lock still prevents Updates and Deletes of the row, but as a weaker lock type, allows for greater read concurrency.

How do you find lock conflicts?

Finding Lock Conflicts

In the Citus article *PostgreSQL rocks, except when it blocks: Understanding locks*,[21] SQL operations are put into a table along with lock types and conflicts. When operations conflict with each other, a red X is placed on the intersection. When they do not conflict, a checkmark is placed in the intersecting table cell.

For example, a SELECT will never conflict with a SELECT, so that intersecting table cell gets a green checkmark.

A SELECT *will* conflict with an ALTER TABLE operation, because the ALTER TABLE requires a table level exclusive lock to run, which blocks concurrent reads. Reads are blocked until the ALTER TABLE lock expires which is when the table modification completes.

Each statement acquires a lock type, usually implicitly.

Next, look at conflicts from CREATE INDEX. Scan the CREATE INDEX column, identifying SQL statements that conflict. Any DML operation will require a lock type that conflicts with the lock type a CREATE INDEX takes. This is why creating

20. https://shiroyasha.io/selecting-for-share-and-update-in-postgresql.html
21. https://www.citusdata.com/blog/2018/02/15/when-postgresql-blocks/